
Legacy Lift
Legacy Lift
LEGACY LIFT
aws
Legacy Lift
AI
Yesterday's Problem → Tomorrow's Solution

Table of Contents

- Meet the Team
- Executive Summary
- High Level Overview
- Development Tools
- Deliverables
- Work Breakdown Structure
- Comparisons
  - Code Comparison - Cobol
  - Code Comparison - Python
  - Infrastructure Comparison
  - RWP and Legacy Lift
- Current Process Flow
- Proposed Solution Flow
- Solution Statement
- Features
- Limitations
- Competition Matrix
- Risks
  - Legal Risk and impacts
  - Risk Analysis and mitigation
- Major functional components
  - COTS and legacy software
- Agile Development Model
- Algorithms
- Database Schema
- Monetization
- Libraries and Frameworks
- RAG
  - What it is
  - How it works
  - Why it matters
- Major Functional Components Diagram
- Data Flow Diagram
- Project GUI Mockup
- Front end Framework
- Conclusion
- References
- Appendix - User Stories

Meet the Team

Spencer Peloquin - He is enrolled in the BSCS MS program at Old Dominion University. He has completed a technical internship at Tacoma Public Utilities. In addition, he has experience in Python, Java, Ruby, C++, C#, SQL and Fortran. He lives in Tacoma WA, where he enjoys running, reading and hiking.



Darius Itam - He is enrolled in the bachelors computer science program as a senior at ODU main campus in Norfolk, VA. He is proficient in Python, Java, JavaScript, and C#. He loves fitness, esports, and music.



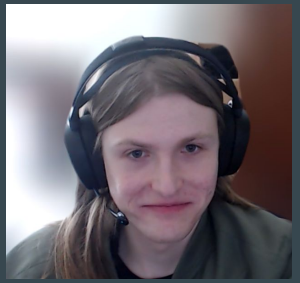


Aaditya Ghimire(AG)- He is enrolled in Computer Science Program at ODU. He has experience in Java + Spring Boot and with main focus on Python deep learning computer vision.



Justin Tendilla - CS Major. Loves playing guitar, video games, and movies. Experienced in C++ and Java.

Jordan Kemelhar - IT/Azure cloud professional and Senior CS student at Old Dominion University. Certified in CyberSecurity Has experience in Java and Azure user management. Hobbies video games, and, hiking, board games.



Executive Summary
AI Pipeline for Legacy Code Modernization



- One major problem the financial industry is facing today is that critical infrastructure is still heavily dependent on COBOL code.
- The main issue is that legacy systems introduce security risks and large amounts of technical debt. Modernizing these systems is extremely difficult because these systems contain complex dependencies and decades of accumulated business logic.
- It is especially important to port code to modern platforms for use in cloud applications, which are primarily incompatible with legacy systems.
- Legacy Lift is an AI-driven pipeline that analyzes dependencies and incrementally converts legacy systems to modern infrastructure. It maintains full compatibility with the legacy code infrastructure while enabling the security and performance benefits of modern programming languages.

High Level Overview

Purpose:

- Translate legacy code from COBOL to modern Python 3.12
- Solve technical debt with legacy codebases
- Resolve ongoing security and memory issues with legacy systems

Mechanics:

- Translates legacy plugin dependencies into modern package hooks
- Preserves class structure from original code

Benefits:

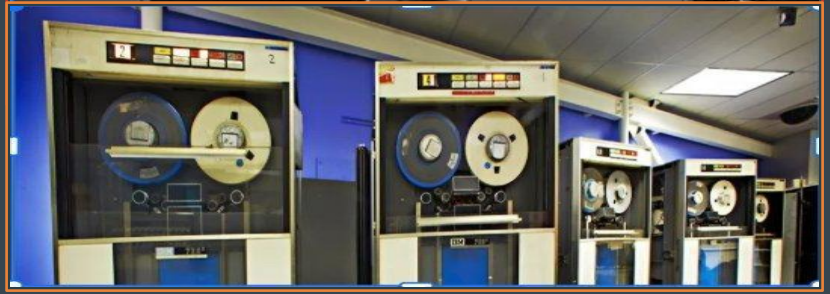
- Functional equivalent code conversion
- Leverages both Generative AI and human review
- Provides an on-ramp to modern systems

The Legacy Hardware

COBOL ran on all three — the language outlived the machines

Before — 1960s







60s

IBM System/360

Engineers at the IBM System/360 operator room. Crew-cut engineers, magnetic tape reels, punch-card input. This is where enterprise computing — and COBOL — was born.

70s

Storage — 1970s

IBM magnetic tape drives — each reel held ~45 MB. No cloud. No instant access. If you needed data, someone physically mounted a reel.

Magnetic tape drives

2022

IBM z16

200 configurable cores, quantum-safe cryptography, AI acceleration on every chip. Same COBOL code. The hardware evolved dramatically. The software didn't.

95% of U.S. ATM transactions processed via COBOL **60%+** of U.S. hospitals run critical systems on legacy code **That's the problem Legacy Lift solves.**

Project Monetization

Discovery and Assessment

Initial evaluation costs range from $2,000 to $10,000, providing tailored project insights for clients.

Framework Implementation

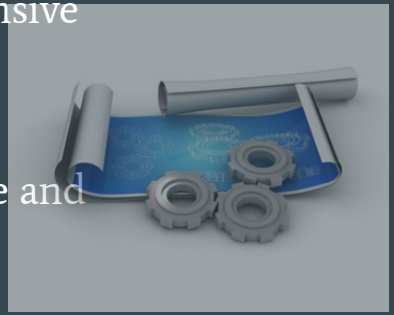
Implementation services are priced $5,000 to $20,000, reflecting increased complexity and customization.

Full Migration

Comprehensive migration services cost $20,000 to $80,000, covering extensive technical work and support.

Annual Maintenance

Ongoing maintenance is $2,000 per year, ensuring continued performance and reliability for clients.



Code Comparison- Cobol to Python

INPUT

COBOL files

```
IDENTIFICATION DIVISION.
PROGRAM-ID. HELLO-TEAM.
DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NAME PIC X(20) VALUE "LegacyLift".
01 WS-COUNT PIC 9(3) VALUE 1.
PROCEDURE DIVISION.
DISPLAY "Hello, " WS-NAME.
ADD 1 TO WS-COUNT.
DISPLAY "Demo count: " WS-COUNT.
STOP RUN.
```

OUTPUT

Python files

hello_team.py

```
# Draft conversion of HELLO-TEAM. Review TODO lines before use.
def main():
    ws_name = 'LegacyLift'
    ws_count = 1
    print('Hello, ', ws_name, sep='')
    ws_count += 1
    print('Demo count: ', ws_count, sep='')
    return


if __name__ == '__main__':
    main()
```

Infrastructure Comparison

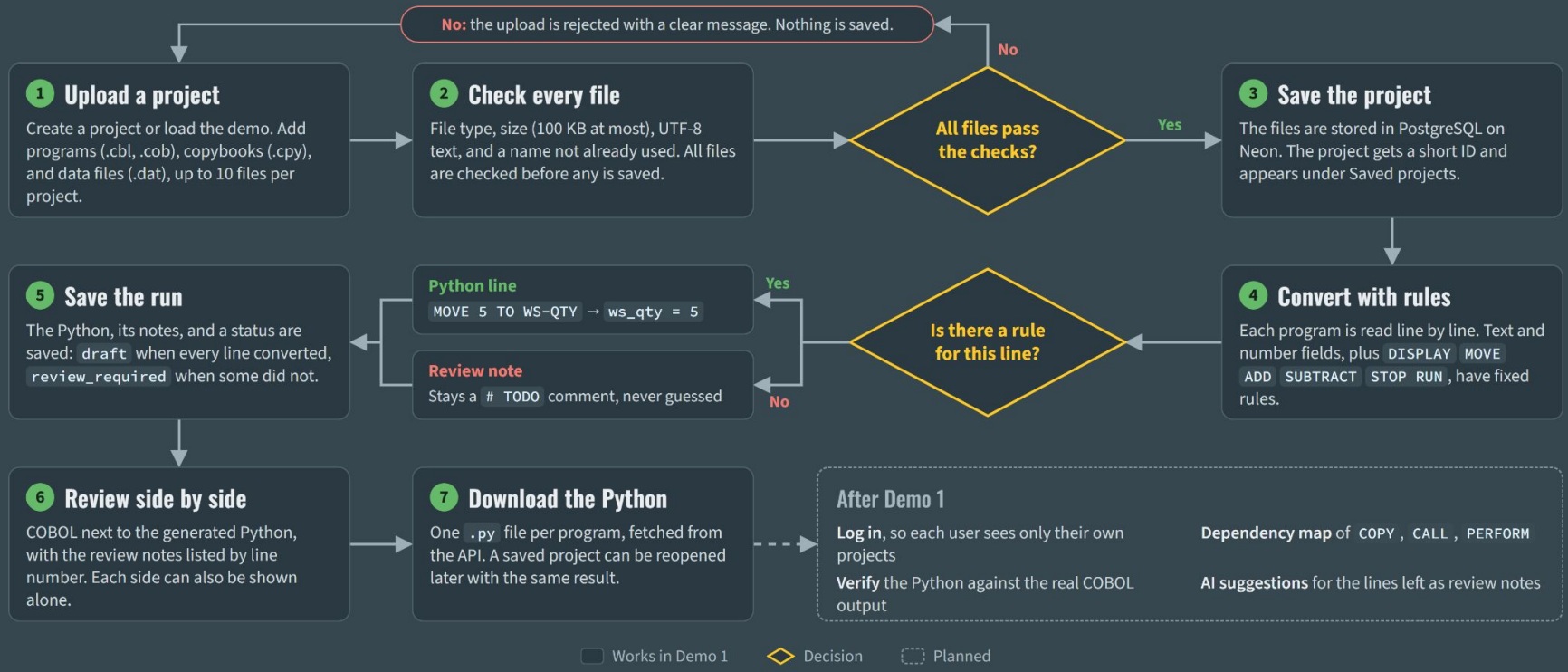
Legacy COBOL environment vs. Modern Python — why the gap matters

Feature	Legacy Code (COBOL/Ada)	Modern Python Environment
Language	COBOL, Fortran, Ada	Python 3.11+
Developers	Scarce — shrinking talent pool	Millions: most taught globally
IDE / Tooling	Proprietary, limited debuggers	VSCode, linters, extensions
Testing	Manual, minimal frameworks	PyTest, Automated CI pipelines
Documentation	Often missing or non-standard	Docstrings, auto-generated docs
Version Control	None or proprietary systems	Git + GitHub + GitHub Actions
Security	Slow patches, legacy vulnerabilities	Active community, rapid patching
Maintenance Cost	High: Specialist rates ($150+/hr)	Standard: Competitive market rates

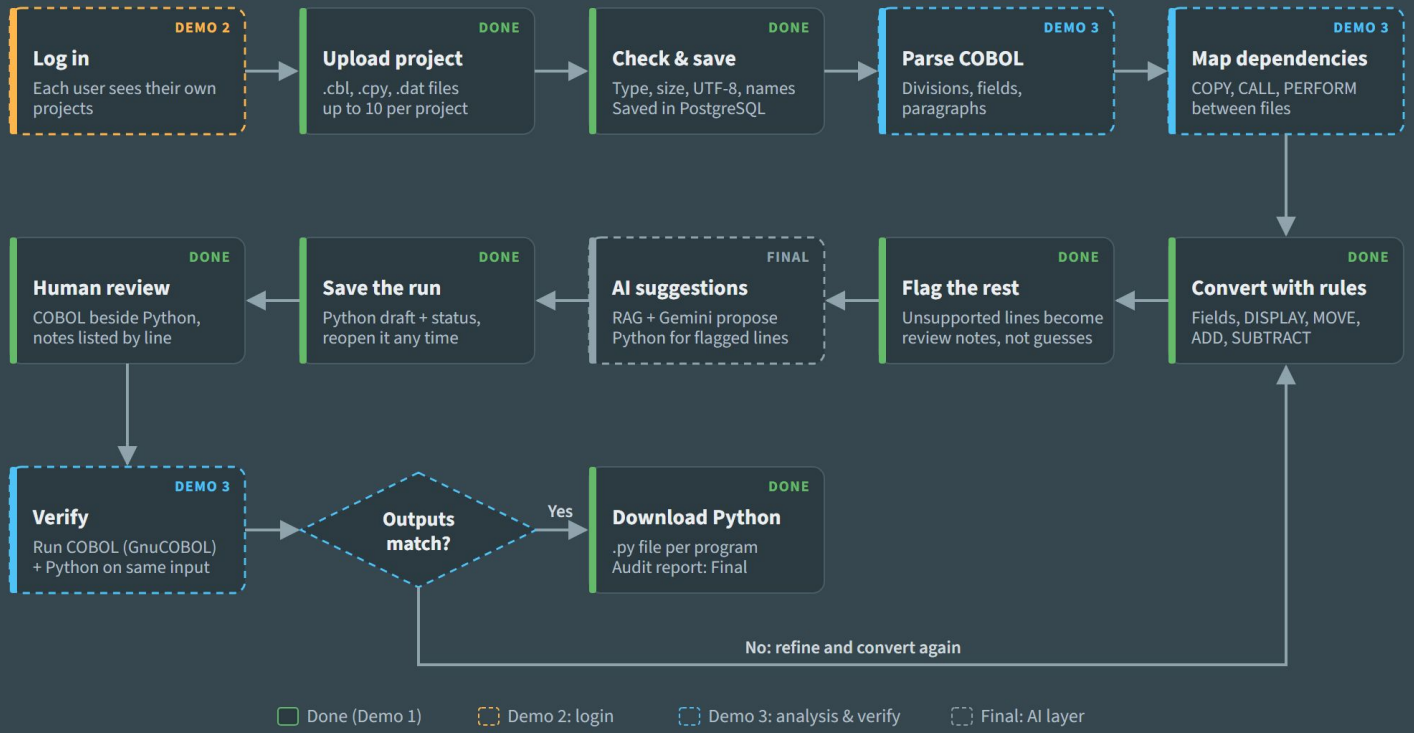
Feature Matrix: RWP vs Legacy Lift

Feature	RWP	Legacy Lift
COBOL → Python 3.11 Translation	✓	✓
Fortran → Python Translation	✓	—
Ada → Python Translation	✓	—
Recreate Original Class Structure	✓	—
Modern Design Refactoring	✓	✓
Runs on Modern x86 Systems	✓	✓
Plugin / Framework Compatibility	✓	—
Preserve Comments & Documentation	✓	✓
Audit Logging	✓	—
Functional Equivalence	✓	✓
Dependency Analysis	✓	✓
Multi-file / Large Codebase	✓	—
CI/CD Integration (Enterprise)	✓	✓
Security Vulnerability Scanning	✓	—

Current Process Flow



Solution Process Flow



Major Functional Components

Tech stack:

Frontend - React + Vite

API - Python 3.12 + FastAPI

Database - PostgreSQL (Neon)

Hosting - GitHub Pages + Render

CI - GitHub Actions

Web app -> Upload, Convert, and Export screens; shows COBOL beside the Python

API layer -> Routes for projects, files, conversion runs, and downloads

Upload checker -> Checks file type, size, UTF-8 text, and names; up to 10 files per project

Project service -> Runs each step and saves it as a run that can be reopened

COBOL converter -> Translates fields and DISPLAY, MOVE, ADD, SUBTRACT into Python using fixed rules

Review notes -> Lines the converter can't translate are kept as notes for a person, never guessed

Storage -> Projects, files, and generated Python saved in PostgreSQL (Neon)

Output check -> Runs the generated Python and compares what it prints

Coming next -> Login (Demo 2); parser, dependency analysis, and GnuCOBOL verify (Demo 3); Gemini + RAG (Final)

Features

This project will produce a prototype software pipeline that analyzes legacy source code and generates structured outputs describing the code's components and relationships, demonstrating the feasibility of using AI-assisted techniques to support legacy system modernization.

Code Translation - COBOL to Python

System Analysis - LegacyLift trains on client systems

Data Awareness - LegacyLift is compatible with any COBOL data

Modernization & Refactoring - Automatically chooses the safest Python Structure

Reliability & Support - Code is verified before deployment

Limitations - Cannot assume missing functionality or generate entirely new code

Work Breakdown Structure

LegacyLift Development Breakdown · COBOL → Python modernization, from design to final demo

1 Planning & Design

- 1.1 Requirements & user stories — DONE
- 1.2 Competition & risk analysis — DONE
- 1.3 Architecture & database design — DONE
- 1.4 Sample COBOL bank: programs, copybooks, data files — DONE
- 1.5 Login design: accounts, sessions, protected pages — DEMO 2

2 Infrastructure Setup

- 2.1 GitHub repo, branches & pull requests — DONE
- 2.2 CI: tests, lint & type checks on GitHub Actions — DONE
- 2.3 Hosting: GitHub Pages (web) + Render (API) — DONE
- 2.4 Neon PostgreSQL database — DONE
- 2.5 Users table & login secret on Render — DEMO 2
- 2.6 Docker + isolated GnuCOBOL runner — DEMO 3

3 Core Building (Frontend + Backend)

- 3.1 FastAPI project API: create, upload, list, delete — DONE
- 3.2 React app: Upload → Convert → Export — DONE
- 3.3 Saved projects, run history & .py download — DONE
- 3.4 Sign up, log in & log out (API + pages) — DEMO 2
- 3.5 Each user sees only their own projects — DEMO 2
- 3.6 Dependency & verification screens — DEMO 3

4 Conversion & Analysis Engine

- 4.1 Rule-based COBOL → Python converter — DONE
- 4.2 Review notes for lines not yet supported — DONE
- 4.3 COBOL parser: divisions, fields, paragraphs — DEMO 3
- 4.4 Dependency analysis: COPY, CALL, PERFORM — DEMO 3
- 4.5 COBOL-style numbers & decimals — DEMO 3
- 4.6 Groups, tables, loops, conditions & calls — FINAL

5 Testing & Verification

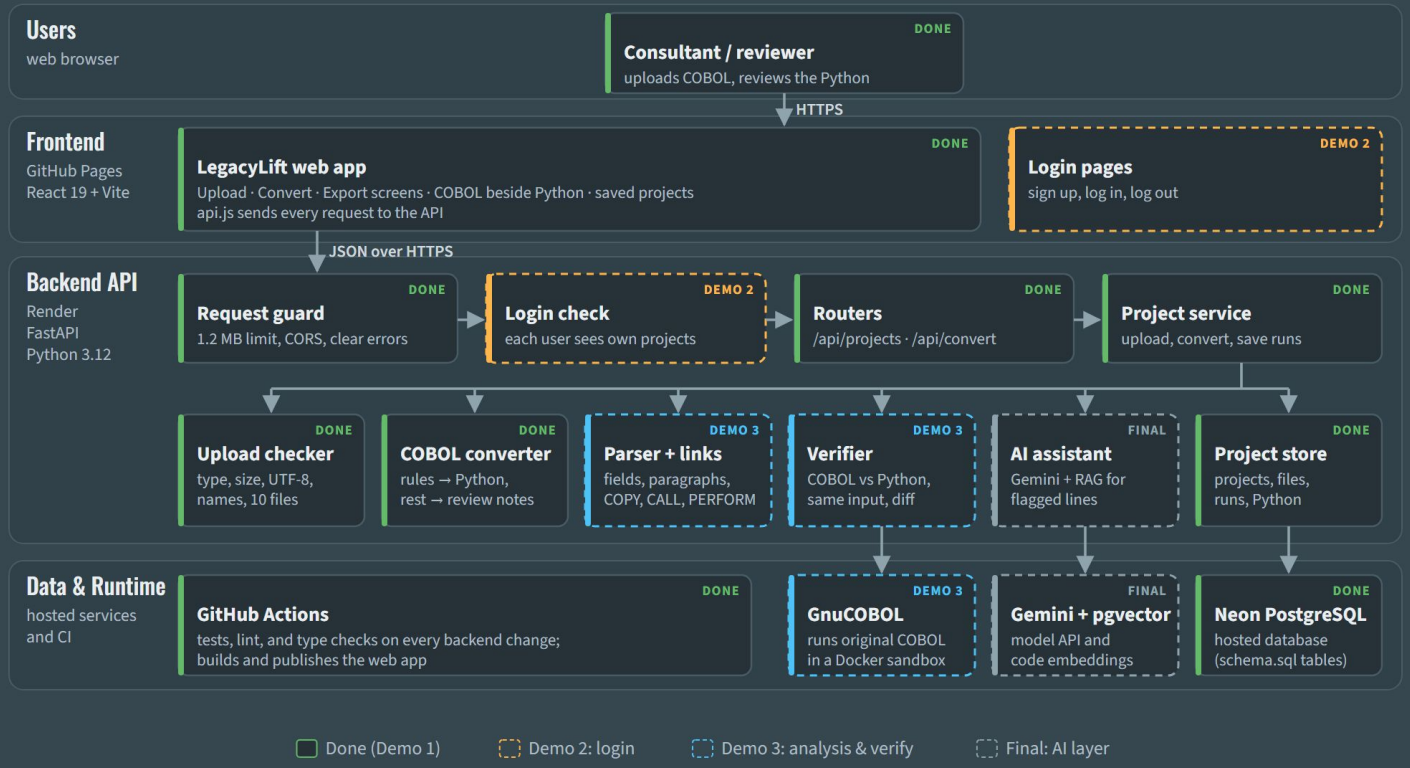
- 5.1 Unit & API tests with pytest (115 passing) — DONE
- 5.2 Run generated Python & check its output — DONE
- 5.3 Login & project-access tests — DEMO 2
- 5.4 Run COBOL (GnuCOBOL) & Python on the same input, compare — DEMO 3
- 5.5 Equivalence tests on the sample bank — DEMO 3

6 AI Integration & Delivery

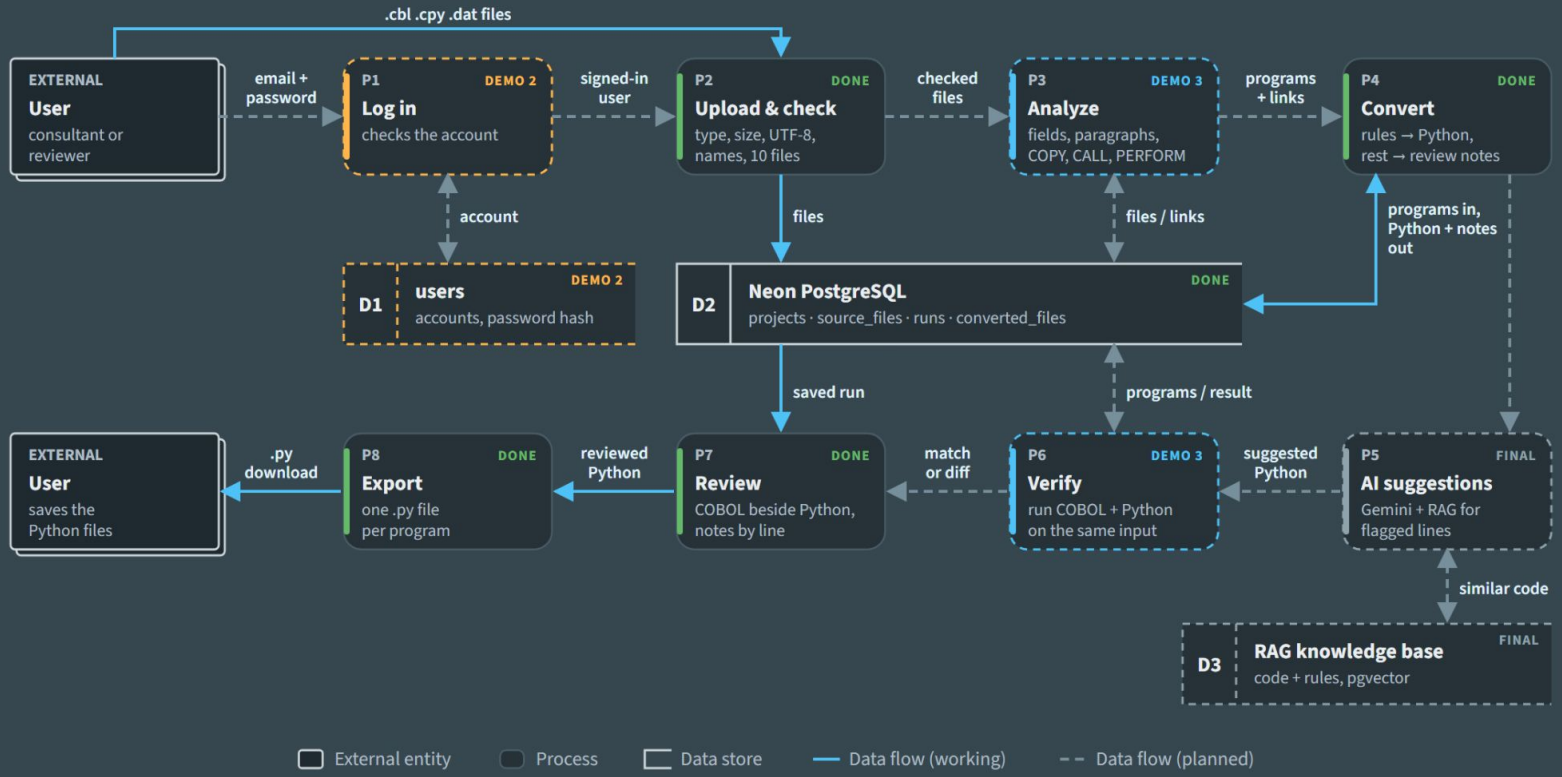
- 6.1 Python download with audit report — FINAL
- 6.2 Gemini translation for unsupported code — FINAL
- 6.3 RAG knowledge base (pgvector) — FINAL
- 6.4 One-command Docker Compose startup — FINAL
- 6.5 Final demo & documentation — FINAL

Done — Demo 1 · **Demo 2** — User login · **Demo 3** — Analysis & verification · **Final** — AI & delivery

Major Functional Components Diagram



Data Flow Diagram



Development Tools



Frontend

- React 19
- Vite 8
- JavaScript
- oxlint

Backend

- Python 3.12
- FastAPI
- Pydantic v2
- Uvicorn

Database

- Neon PostgreSQL
- psycopg 3
- pgvector FINAL

Testing & Quality

- pytest
- ruff lint + format
- mypy strict
- GnuCOBOL DEMO 3

Version Control

- Git
- GitHub
- GitHub Actions CI
- Pull requests

Hosting

- GitHub Pages web app
- Render API
- Neon database
- Docker DEMO 3

Dev Environment

- VS Code
- uv packages
- Node.js 24
- Windows 11 / Ubuntu

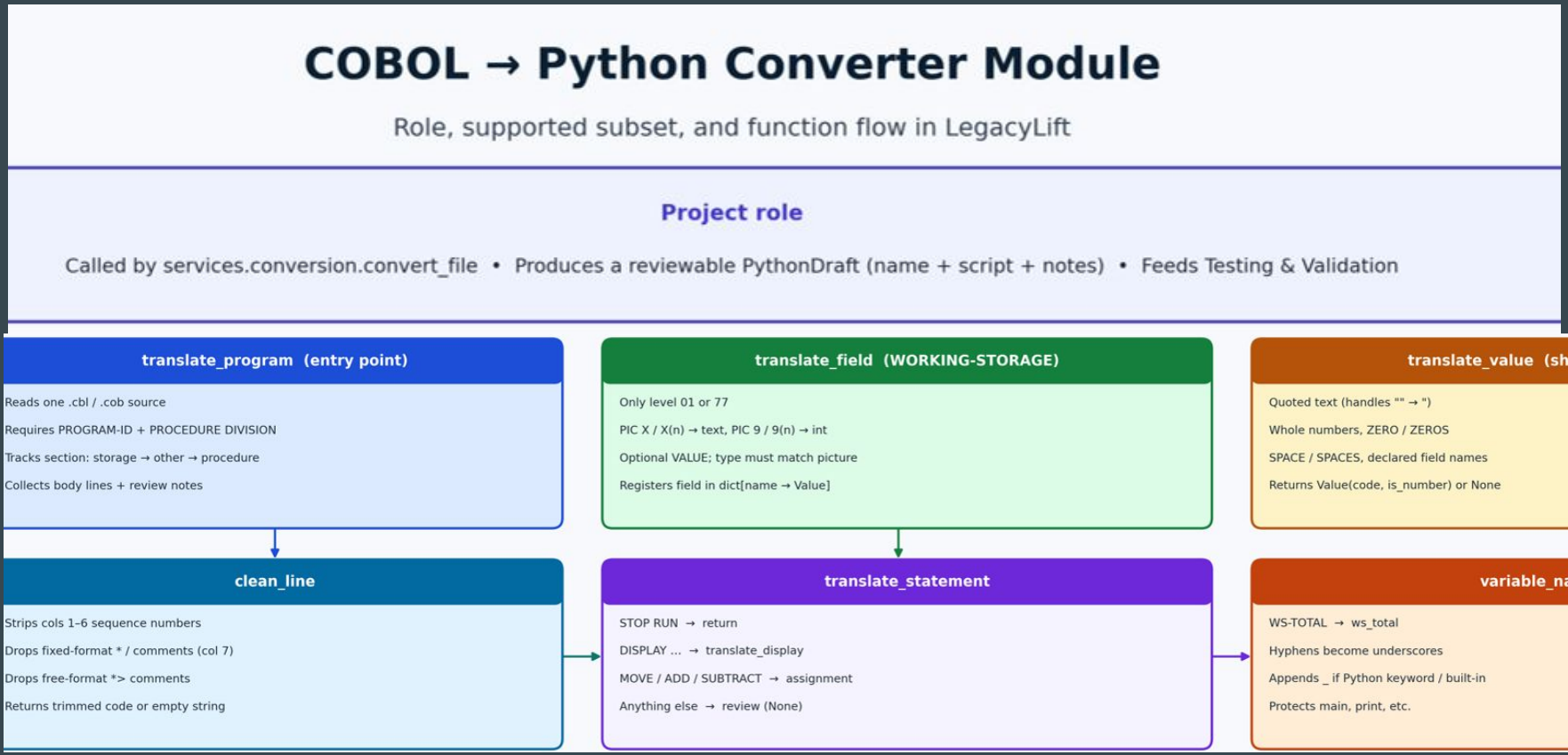
AI Layer

- Gemini FINAL
- RAG retrieval FINAL

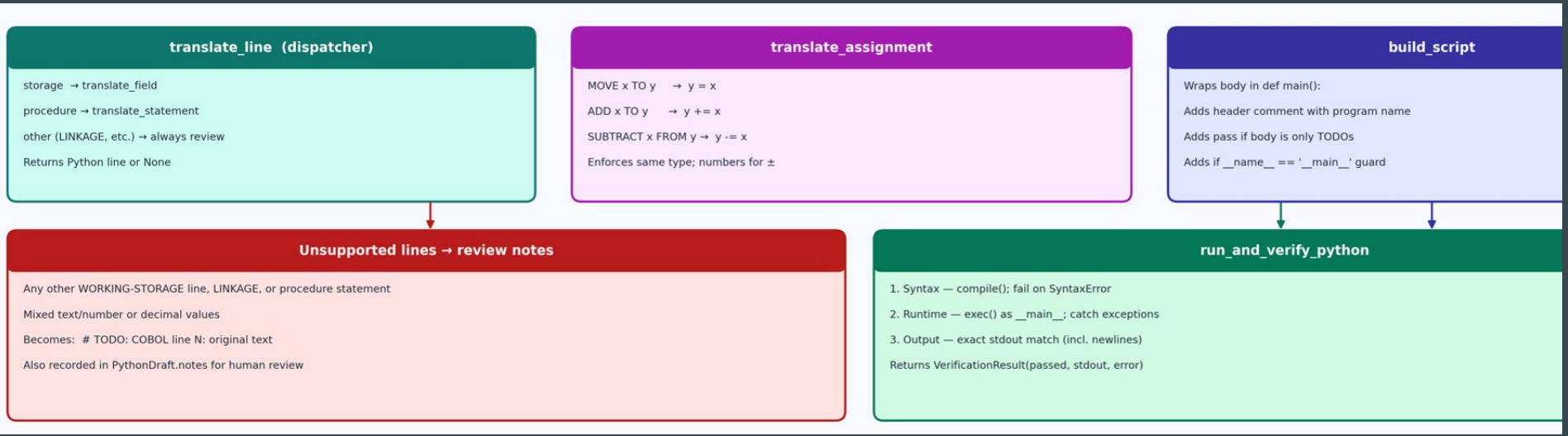
How it connects: **React app** GitHub Pages → **FastAPI** Render → **PostgreSQL** Neon

In use (Demo 1) · Planned for Demo 3 · Planned for Final

Backend - Converter



Backend - Converter



Supported subset (everything else becomes a review note)

WORKING-STORAGE

- Level 01 or 77 fields only
- PIC X, XX, X(n) → text (default '')
- PIC 9, 99, 9(n) → whole numbers (default 0)
- Optional VALUE; text may use doubled quotes

PROCEDURE DIVISION

- DISPLAY items → print(..., sep='')
- MOVE ... TO → assignment
- ADD ... TO / SUBTRACT ... FROM
- STOP RUN → return

Safety rules

- No type mismatches generated
- Decimals and expressions → review
- LINKAGE and other sections → review
- Output is a draft, not production code

API DATA FLOW

LegacyLift REST API · FastAPI · JSON over HTTPS · `legacylift-api.onrender.com` · try every route at `/docs`

Projects — `/api/projects`

GET	(base path)	list projects, newest first
POST	(base path)	create from {"name"} → 201
POST	/demo	project with sample COBOL
GET	/{id}	project and its files
DELETE	/{id}	delete project → 204

Files — `/api/projects`

POST	/{id}/files	upload (multipart "files")
DELETE	/{id}/files/{file_id}	remove one file → 204

Pipeline steps — `/api/projects`

POST	/{id}/analyze	file counts today · DEMO 3 links
POST	/{id}/convert	Python + review notes, saved as a run
POST	/{id}/verify	placeholder · DEMO 3 COBOL vs Python

POST /api/projects/1/convert → 200 (real response, Python shortened)

```
{ "project_id": "1", "run_id": "1", "status": "draft",
  "files": [{
    "source_name": "store_report.cbl",  "python_name": "store_report.py",
    "program_name": "STORE-REPORT",  "source_file_id": "1",
    "python": "# Draft conversion of STORE-REPORT. ...",
    "notes": [],  "status": "draft"  // or "review_required"
  }] }
```

Runs & downloads — `/api/projects`

GET	/{id}/runs	run history
GET	/{id}/runs/{run_id}	one run with its Python
GET	/{id}/runs/{run_id}/files/{name}	download .py

Quick convert (no project) — `/api/convert`

POST	(base path)	convert 1–5 COBOL files
GET	/{download_id}/files/{name}	download .py

Health

GET	/health	{"status": "ok"}

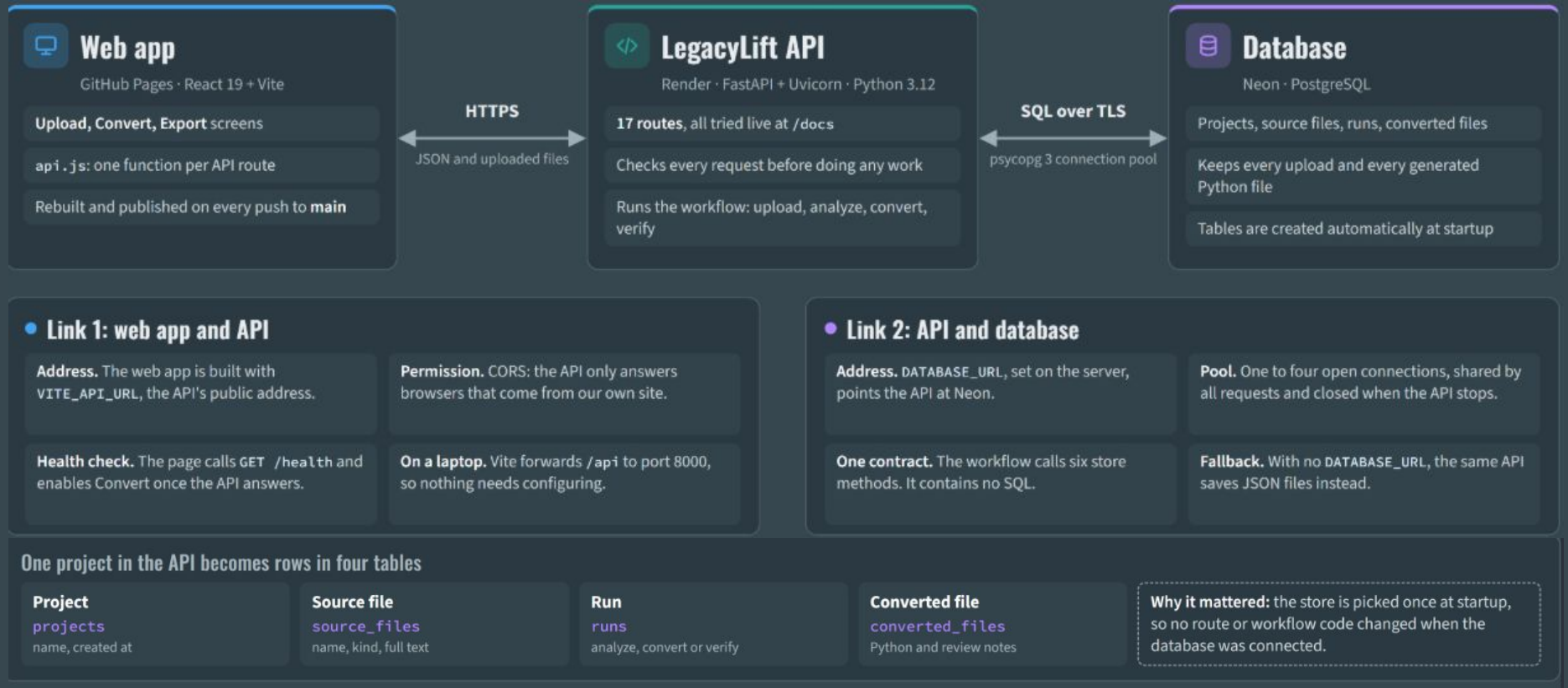
Login DEMO 2 — `/api/auth`

POST	/register	sign up
POST	/login	log in
GET	/me	who is logged in

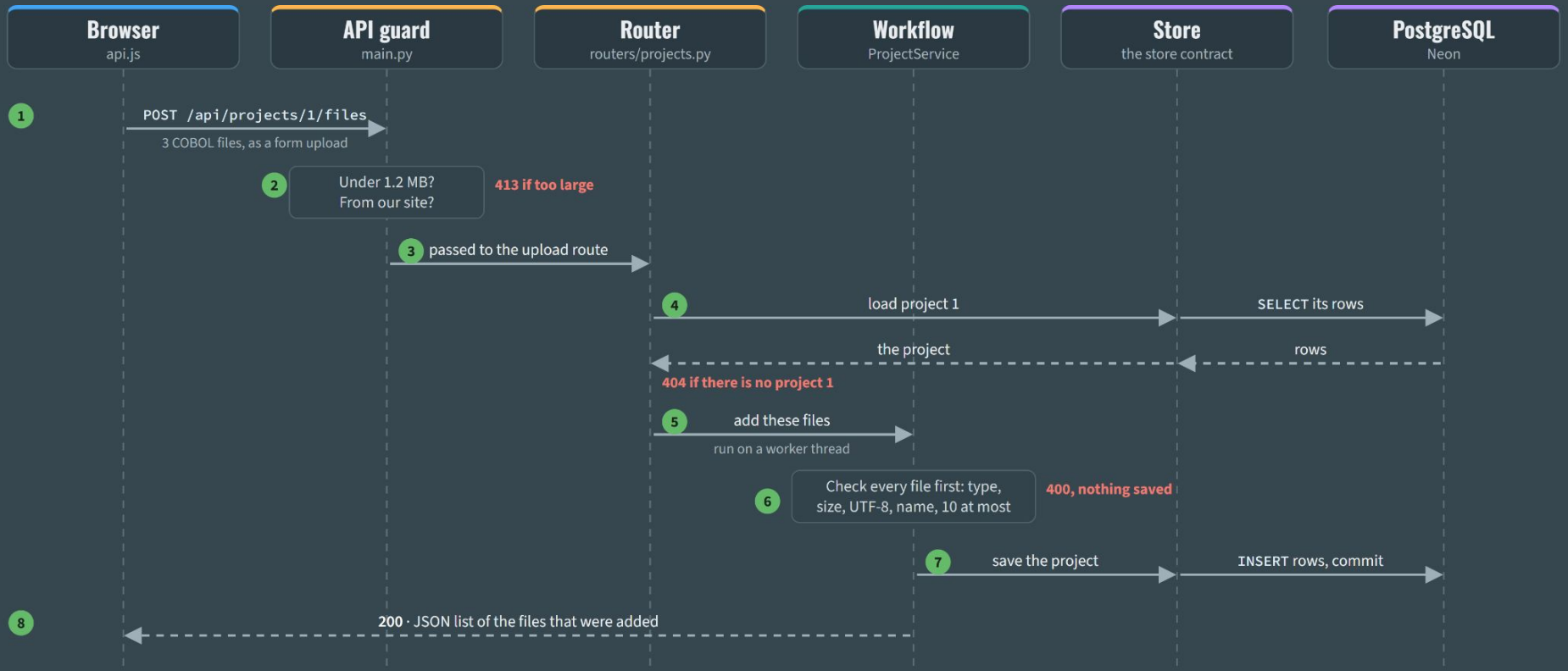
Errors are always `{"detail": "Project not found."}`

400	bad file, name, or step run too early
404	unknown project, file, or run
413	request over 1.2 MB
422	missing or invalid field

API REQUESTS

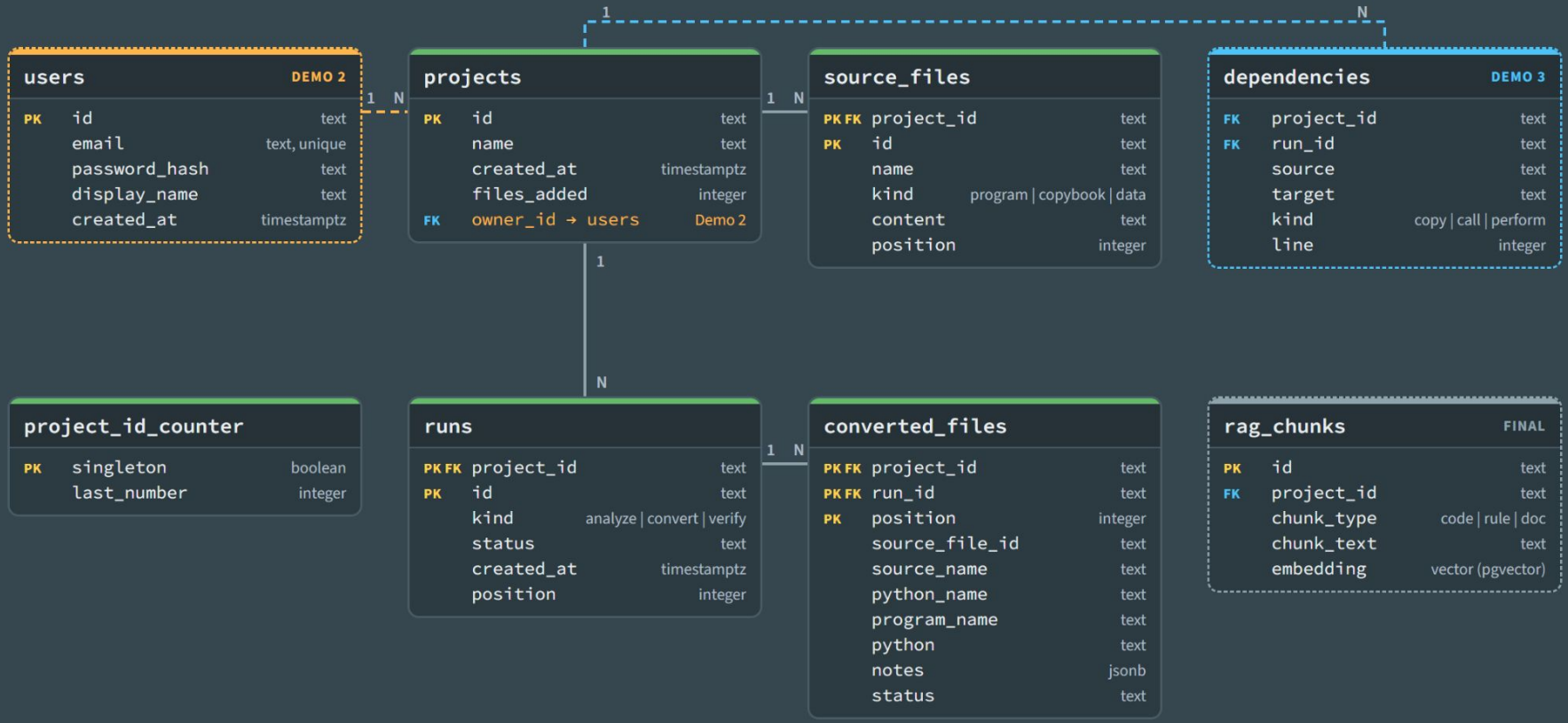


API SEQUENCE

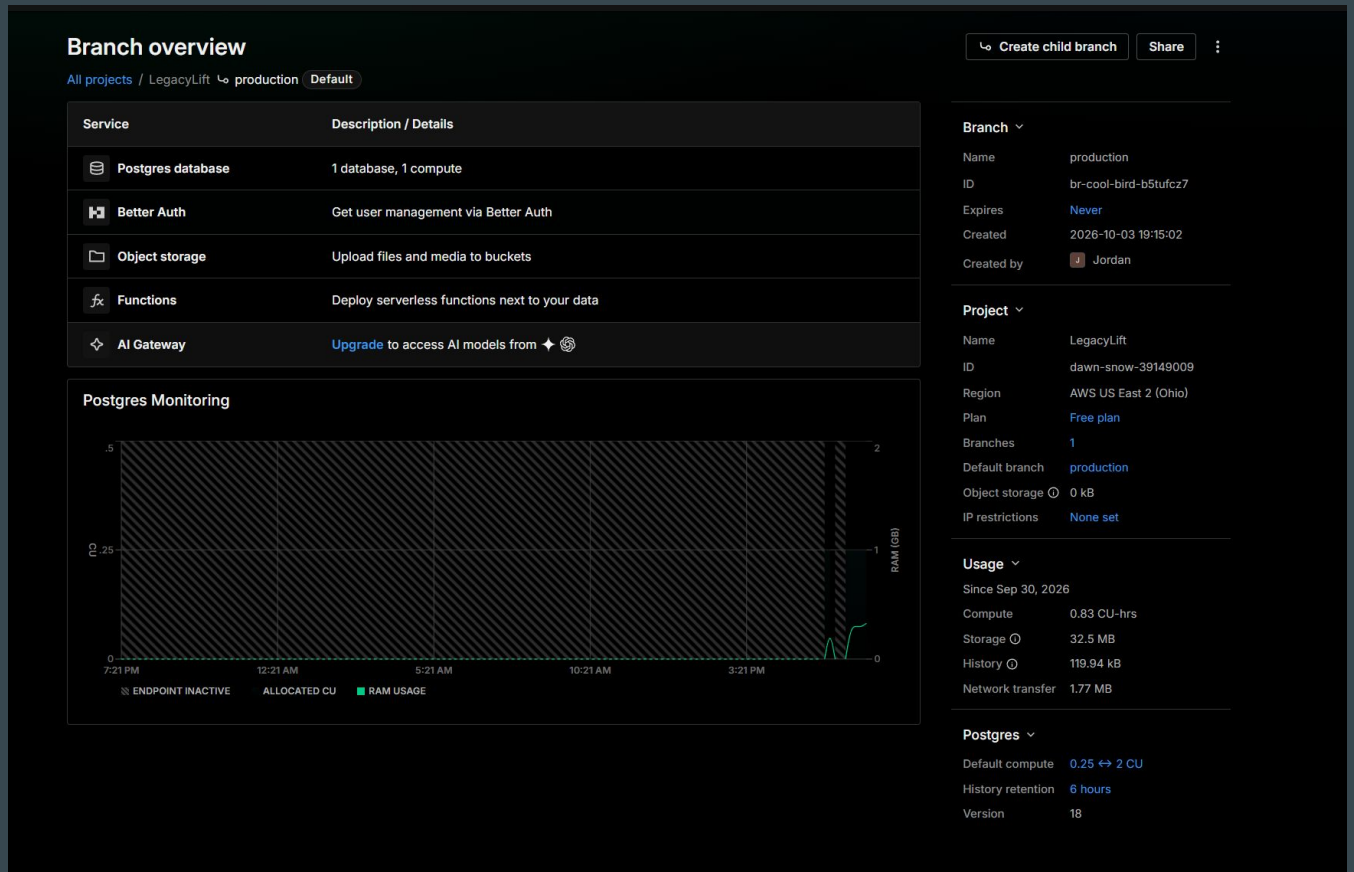


Every route takes this path: **guard → router → workflow → store**. Only the workflow step changes.

Database Schema



Dashboard Overview Neon



Current Tables

Current functional tables include projects, runs, source files, and converted files.

PROJECTS:

#	id	name	created_at	files_added
1	8	LegacyLift demo	2026-10-04 15:49:25.906213+00	3
2	9	TestOpen	2026-10-04 15:49:40.750164+00	0
3	10	LegacyLift demo	2026-10-04 20:10:13.777927+00	3
4	12	LegacyLift demo	2026-10-11 22:09:30.571169+00	3
5	13	LegacyLift demo	2026-10-11 22:09:31.346957+00	3
6	14	LegacyLift demo	2026-10-11 22:09:32.021297+00	3
7	15	LegacyLift demo	2026-10-11 22:09:32.628516+00	3

RUNS:

#	project_id	id	kind	status	created_at	position
1	10	1	convert	draft	2026-10-04 20:10:25.618863+00	0
2	10	2	convert	draft	2026-10-04 20:10:46.059137+00	1
3	10	3	convert	draft	2026-10-04 20:10:54.940415+00	2

SOURCE_FILES:

#	project_id	id	name	kind	content	position
1	8	1	store_report.cbl	program	IDENTIFICATION DIVISION. PROGRAM-ID. STORE-REPORT. DATA DIVISION. WORKING-STORAGE SECTION. 01 WS-STORE PIC X(20) VALUE "LegacyLift Market". 01 WS-CUSTOMER PIC X(20) VALUE "Ada". 01 WS-ITEM PIC X(20) VALUE "Coffee". 01 WS-ITEM-PRICE PIC 9(3) VALUE 12. 01 WS-SHIPPING PIC 9(3) VALUE 5. 01 WS-DISCOUNT PIC 9(3) VALUE 3. 01 WS-TOTAL PIC 9(4) VALUE 0. 01 WS-ORDER-NUMBER PIC 9(4) VALUE 1042. PROCEDURE DIVISION. DISPLAY "=== " WS-STORE " ===". DISPLAY "Order #: " WS-ORDER-NUMBER. DISPLAY "Customer: " WS-CUSTOMER. DISPLAY "Item: " WS-ITEM. DISPLAY "Price: $" WS-ITEM-PRICE. DISPLAY "Shipping: $" WS-SHIPPING. DISPLAY "Discount: $" WS-DISCOUNT. MOVE WS-ITEM-PRICE TO WS-TOTAL. ADD WS-SHIPPING TO WS-TOTAL. SUBTRACT WS-DISCOUNT FROM WS-TOTAL. DISPLAY "Amount due: $" WS-TOTAL. DISPLAY "Thanks for your order!". STOP RUN.	0
2	8	2	order.cpy	copybook	01 ORDER-RECORD. 05 ORDER-NUMBER PIC 9(4). 05 ORDER-CUSTOMER PIC X(10). 05 ORDER-ITEM PIC X(10).	1
3	8	3	orders.dat	data	1042Ada Coffee	2
4	10	1	store_report.cbl	program	IDENTIFICATION DIVISION. PROGRAM-ID. STORE-REPORT. DATA DIVISION. WORKING-STORAGE SECTION. 01 WS-STORE PIC X(20) VALUE "LegacyLift Market". 01 WS-CUSTOMER PIC X(20) VALUE "Ada". 01 WS-ITEM PIC X(20) VALUE "Coffee". 01 WS-ITEM-PRICE PIC 9(3) VALUE 12. 01 WS-SHIPPING PIC 9(3) VALUE 5. 01 WS-DISCOUNT PIC 9(3) VALUE 3. 01 WS-TOTAL PIC 9(4) VALUE 0. 01 WS-ORDER-NUMBER PIC 9(4) VALUE 1042. PROCEDURE DIVISION. DISPLAY "=== " WS-STORE " ===". DISPLAY "Order #: " WS-ORDER-NUMBER. DISPLAY "Customer: " WS-CUSTOMER. DISPLAY "Item: " WS-ITEM. DISPLAY "Price: $" WS-ITEM-PRICE. DISPLAY "Shipping: $" WS-SHIPPING. DISPLAY "Discount: $" WS-DISCOUNT. MOVE WS-ITEM-PRICE TO WS-TOTAL. ADD WS-SHIPPING TO WS-TOTAL. SUBTRACT WS-DISCOUNT FROM WS-TOTAL. DISPLAY "Amount due: $" WS-TOTAL. DISPLAY "Thanks for your order!". STOP RUN.	0
5	10	2	order.cpy	copybook	01 ORDER-RECORD. 05 ORDER-NUMBER PIC 9(4). 05 ORDER-CUSTOMER PIC X(10). 05 ORDER-ITEM PIC X(10).	1
6	10	3	orders.dat	data	1042Ada Coffee	2
7	12	1	store_report.cbl	program	IDENTIFICATION DIVISION. PROGRAM-ID. STORE-REPORT. DATA DIVISION. WORKING-STORAGE SECTION. 01 WS-STORE PIC X(20) VALUE "LegacyLift Market". 01 WS-CUSTOMER PIC X(20) VALUE "Ada". 01 WS-ITEM PIC X(20) VALUE "Coffee". 01 WS-ITEM-PRICE PIC 9(3) VALUE 12. 01 WS-SHIPPING PIC 9(3) VALUE 5. 01 WS-DISCOUNT PIC 9(3) VALUE 3. 01 WS-TOTAL PIC 9(4) VALUE 0. 01 WS-ORDER-NUMBER PIC 9(4) VALUE 1042. PROCEDURE DIVISION. DISPLAY "=== " WS-STORE " ===". DISPLAY "Order #: " WS-ORDER-NUMBER. DISPLAY "Customer: " WS-CUSTOMER. DISPLAY "Item: " WS-ITEM. DISPLAY "Price: $" WS-ITEM-PRICE. DISPLAY "Shipping: $" WS-SHIPPING. DISPLAY "Discount: $" WS-DISCOUNT. MOVE WS-ITEM-PRICE TO WS-TOTAL. ADD WS-SHIPPING TO WS-TOTAL. SUBTRACT WS-DISCOUNT FROM WS-TOTAL. DISPLAY "Amount due: $" WS-TOTAL. DISPLAY "Thanks for your order!". STOP RUN.	0

CONVERTED_FILES:

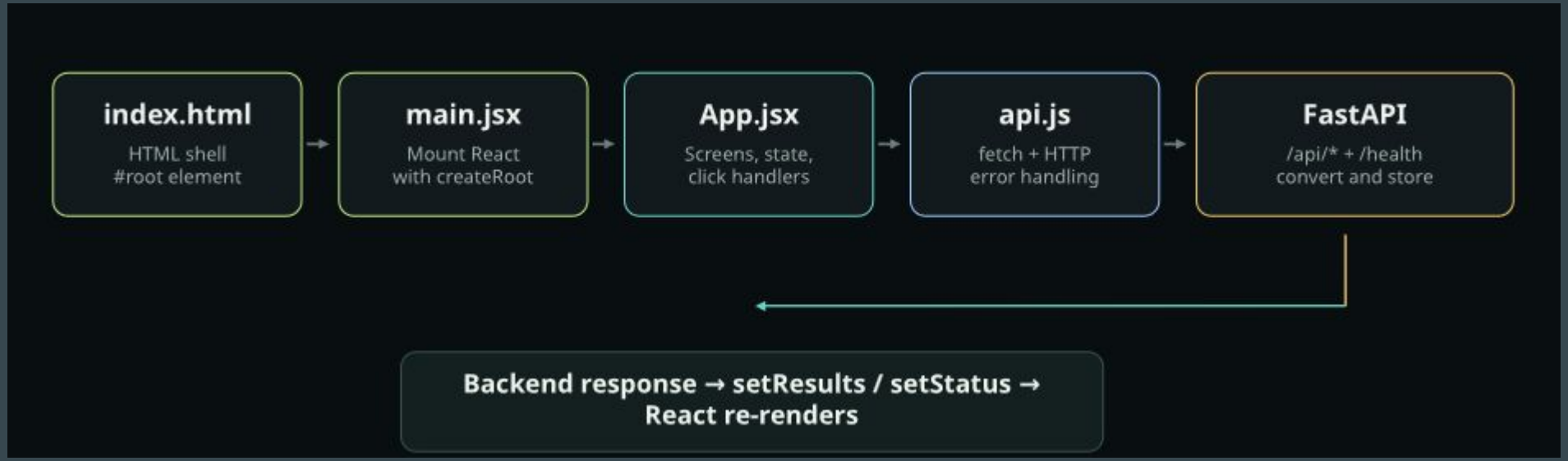
#	project_id	run_id	position	source_file_id	source_name	python_name	program_name	python	notes	status
1	10	1	0	1	store_report.cbl	store_report.py	STORE-REPORT	# Draft conversion of STORE-REPORT. Review TODO lines before use. def main(): ws_store = 'LegacyLift Market' ws_customer = 'Ada' ws_item = 'Coffee' ws_item_price = 12 ws_shipping = 5 ws_discount = 3 ws_total = 0 ws_order_number = 1042 print('=== ', ws_store, ' ===', sep='') print('Order #: ', ws_order_number, sep='') print('Customer: ', ws_customer, sep='') print('Item: ', ws_item, sep='') print('Price: $', ws_item_price, sep='') print('Shipping: $', ws_shipping, sep='') print('Discount: $', ws_discount, sep='') ws_total = ws_item_price ws_total += ws_shipping ws_total -= ws_discount print('Amount due: $', ws_total, sep='') print('Thanks for your order!', sep='') return if __name__ == '__main__': main()	[]	draft
2	10	2	0	1	store_report.cbl	store_report.py	STORE-REPORT	# Draft conversion of STORE-REPORT. Review TODO lines before use. def main(): ws_store = 'LegacyLift Market' ws_customer = 'Ada' ws_item = 'Coffee' ws_item_price = 12 ws_shipping = 5 ws_discount = 3 ws_total = 0 ws_order_number = 1042 print('=== ', ws_store, ' ===', sep='') print('Order #: ', ws_order_number, sep='') print('Customer: ', ws_customer, sep='') print('Item: ', ws_item, sep='') print('Price: $', ws_item_price, sep='') print('Shipping: $', ws_shipping, sep='') print('Discount: $', ws_discount, sep='') ws_total = ws_item_price ws_total += ws_shipping ws_total -= ws_discount print('Amount due: $', ws_total, sep='') print('Thanks for your order!', sep='') return if __name__ == '__main__': main()	[]	draft

Live DEMO

Front end framework

- React + Vite: App.jsx is the screen, while api.js is every call to the API.
- Locally, Vite forwards /api to the API on port 8000. The hosted site calls Render
- Each button calls one API address. Open calls three: the project, its runs, then the saved Python.
- A bad file is stopped on the screen and named. If the API rejects it anyways, a red box shows the API’s message.

Front end path abstraction



Follow the data from the browser entry point to the backend and back

Front end Tools



Style and motion

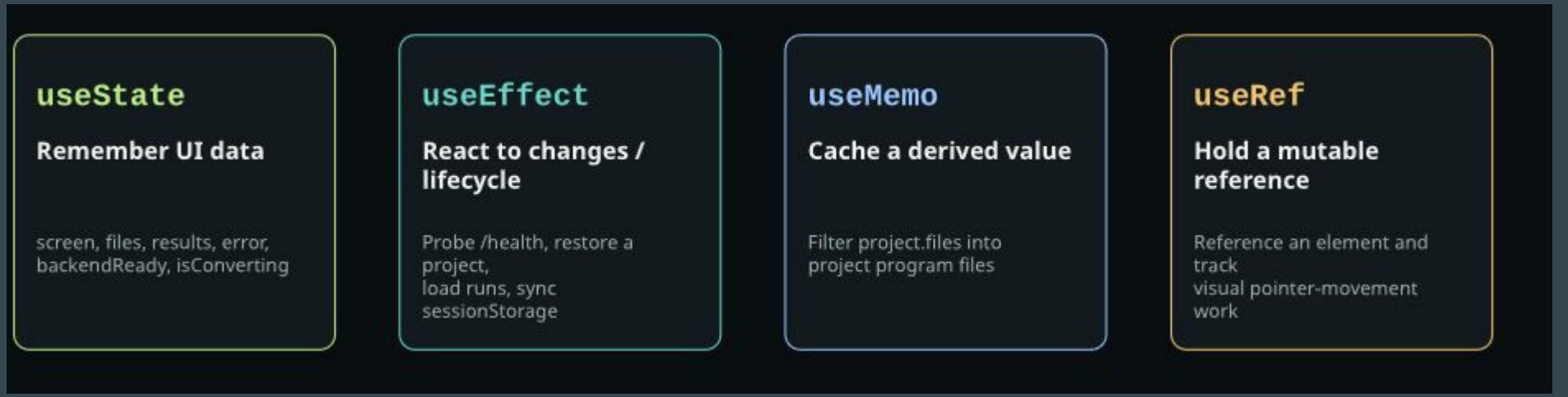
-Background.jsx:

- controls builds deterministic positions for 42 particles and 16 network nodes
- Renders grids, orbits, beams, scanline and vignette layers.
- Tracks pointer movement and writes CSS variables for subtle motion.

-CSS files:

- Global font, black page background, text defaults and basic form/button rules
- Workspace layout, panels, nav buttons, status badges, errors and responsive behavior

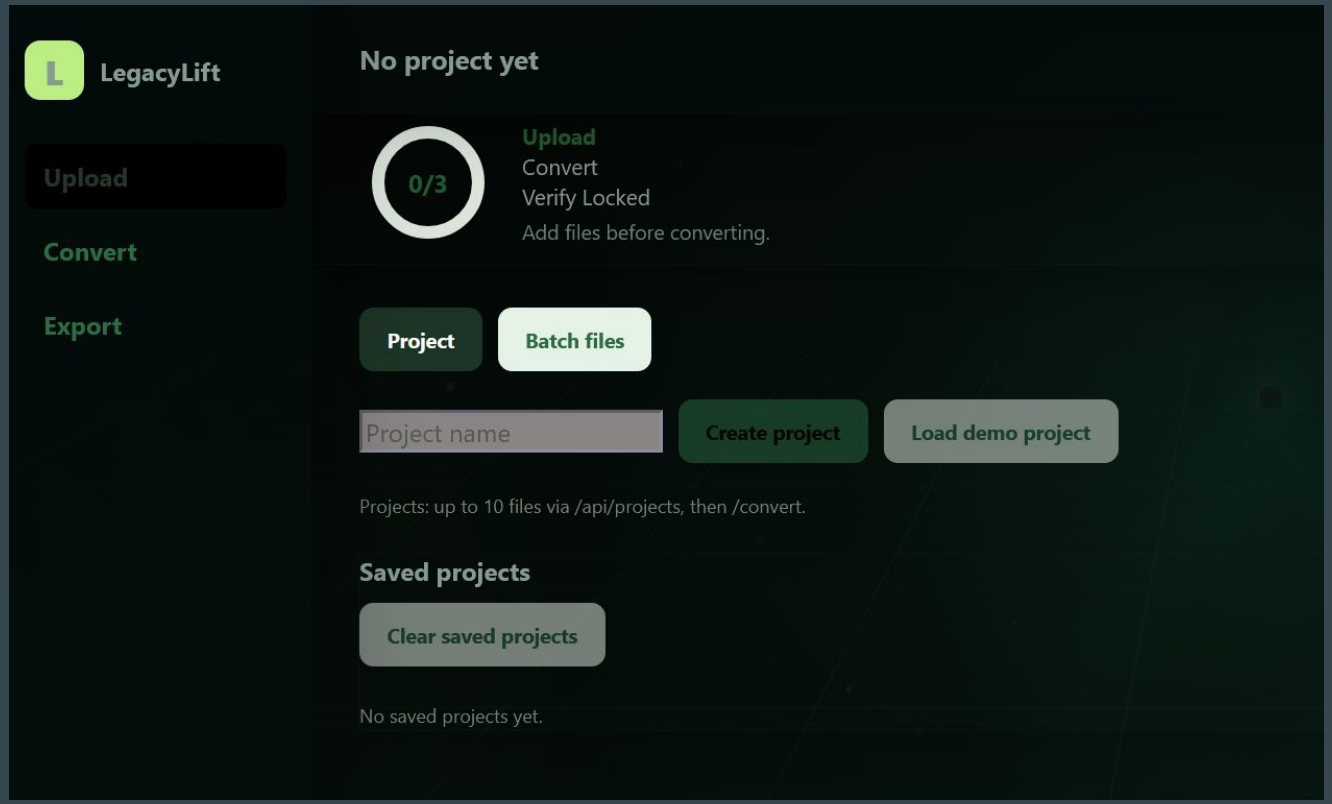
Front end hooks



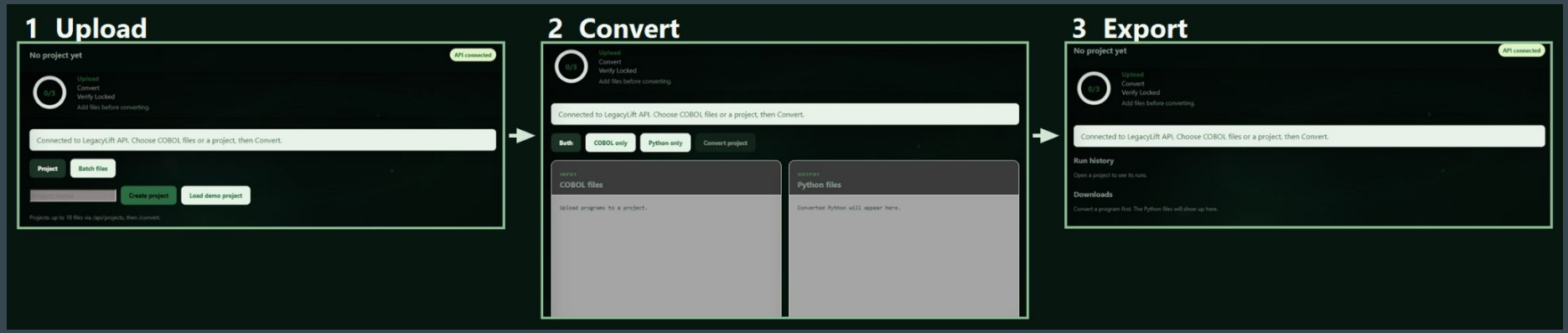
Each hook has a specific job. Read the state names before reading the JSX.

Driver

- Upload: Input + project selection
- Convert: Action + review
- Export: Download + history

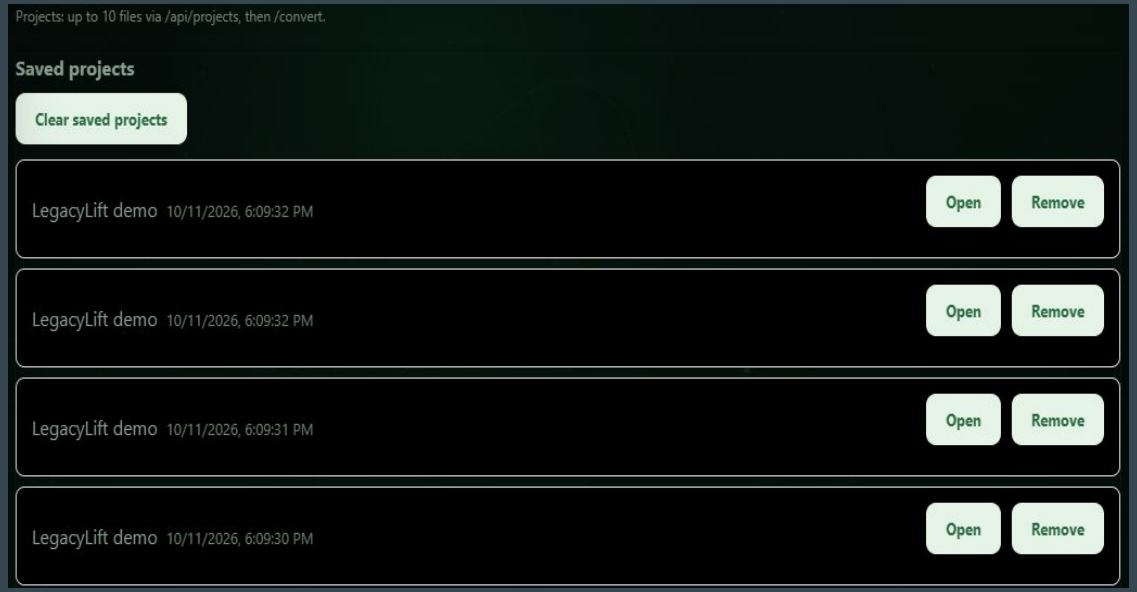


Project Screens (GUI)



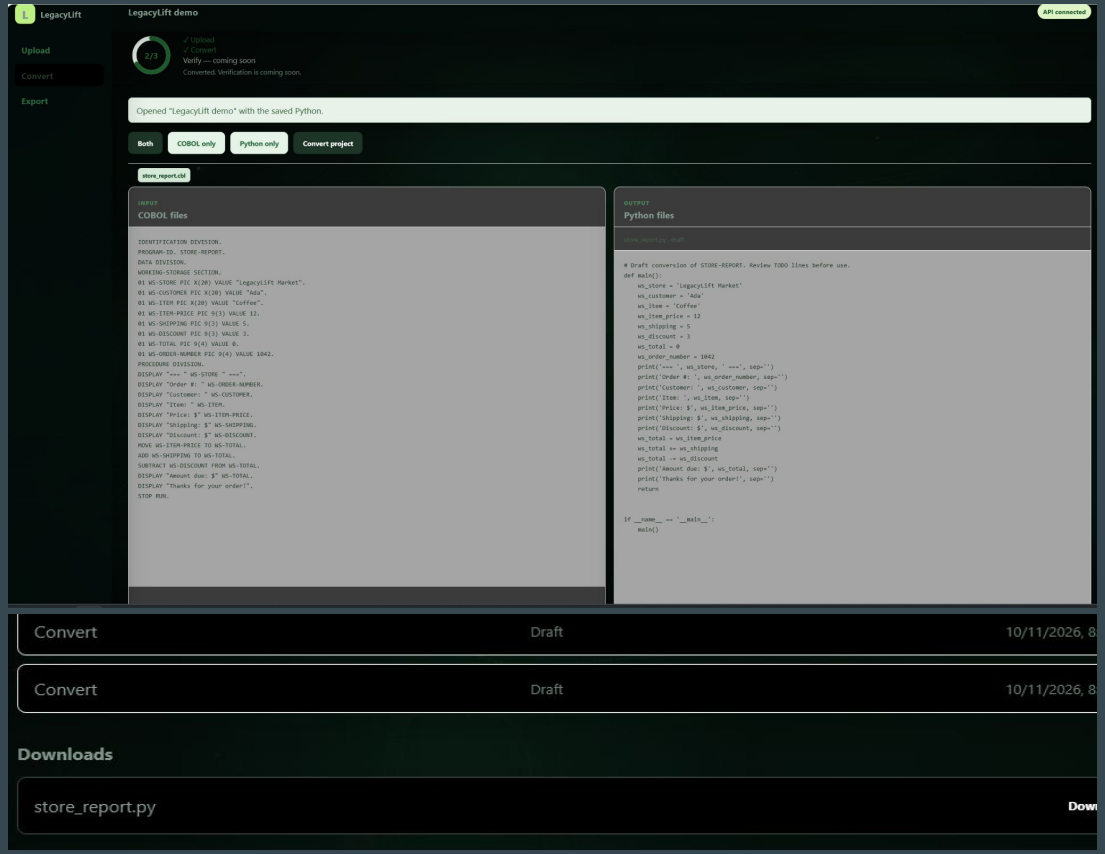
- Three screens: Upload, Convert, Export
- Project saves files on the API. Batch converts in the browser and is not saved
- Convert shows COBOL beside Python, plus lines a developer must review
- Export lists saved runs and downloads as .py files from the API
- Refresh puts the same project and its last translation back on screen

Saved projects



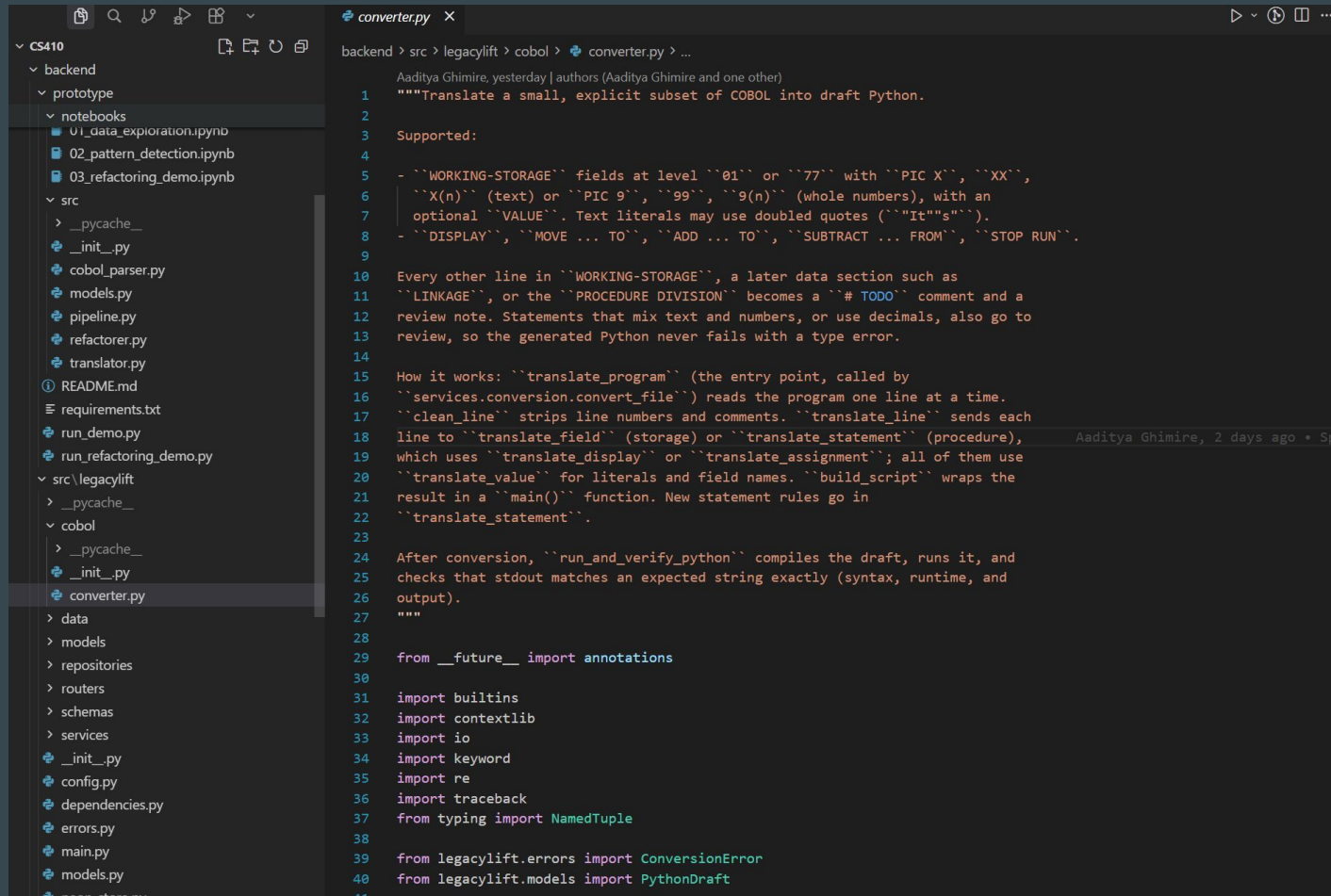
- Upload lists every project stored on the API.
- Open loads that project and its last Python, then switches to Convert.
- Remove deletes one project. Clear deletes every project in the list.
- The project id stays in the browser tab, so a refresh opens the same project.

Output



- COBOL
- Python
- Export Screen: History and .py file

Implementation



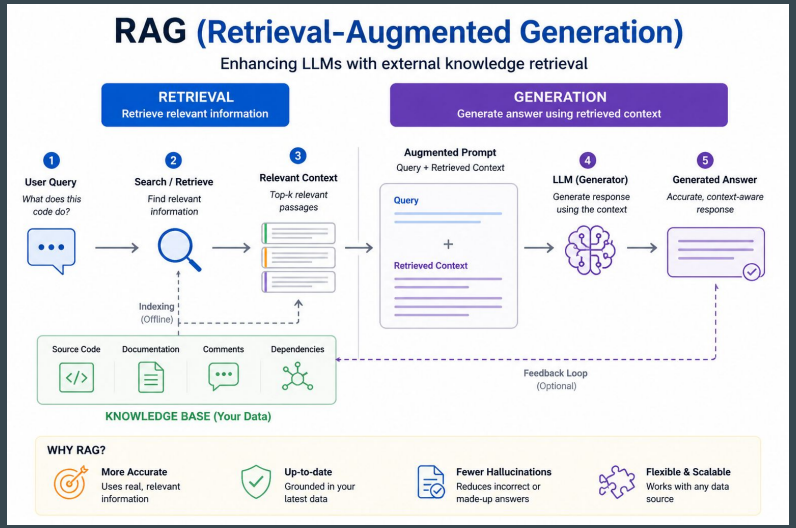
RAG - What It Is

Retrieval-Augmented Generation

- Combines data retrieval + AI generation
- Retrieves relevant code, documentation, and dependencies before AI processing
- Provides context to the model instead of relying on memory alone
- Implemented through Model Context Protocol (MCP), optionally through an external server
- To be implemented partially in Demo 2 and in full by Demo 3

RAG - How It Works

- Legacy code and documentation are split into searchable chunks
- Relevant context is retrieved based on the current task
- Retrieved data is added to the prompt
- Gemini generates analysis or translated code utilizing that context



RAG - Why It Matters for Legacy Lift

- Reduce AI errors
  - Prevents the model from guessing or making up logic
  - Uses real code and context
- Preserves Business Logic
  - Legacy systems contain critical rules spread across multiple files
  - RAG ensures the AI sees related code, dependencies, and comments
- Improves Consistency
  - The framework pulls from the same dataset every time as opposed to what is uploaded at each instance.
- Enables Safer Modernization
  - Reduces risk of breaking critical system behavior during conversion

Conclusion



Legacy Lift is a viable solution for small businesses seeking a stable, cost effective means for updating and modernizing their legacy code infrastructure.

Questions?

References

"12 Best Legacy Modernization Tools for Companies in 2026." *12 Best Legacy Modernization Tools for Companies in 2026*, 2 Mar. 2026, launchpad.io/blog/best-legacy-modernization-tools.

"2025 Legacy Code Stats." *Pragmatic Coders*, 2 Sept. 2025, www.pragmaticcoders.com/resources/legacy-code-stats.

The 2025 State of Legacy Software Modernization Report, www.saritasa.com/wp-content/uploads/2025/07/Saritasa-2025_State_of_Legacy_Software_Modernization_Report.pdf. Accessed 11 Mar. 2026.

"44 Legacy System Modernization Statistics Every Enterprise Should Know in 2026." *DreamFactory [LIVE]*, www.dreamfactory.com/hub/legacy-system-modernization-statistics. Accessed 11 Mar. 2026.

Ellienosrat, et al. "Best Practices for Leveraging Azure Openai in Code Conversion Scenarios: Microsoft Community Hub." *TECHCOMMUNITY.MICROSOFT.COM*, 26 Mar. 2025, techcommunity.microsoft.com/blog/azure-ai-foundry-blog/best-practices-for-leveraging-azure-openai-in-code-conversion-scenarios/4395993.

"IBM Watsonx Code Assistant for Z." *IBM*, 12 Apr. 2024, www.ibm.com/products/watsonx-code-assistant-z.

"Lost in Translation: What the AI Code Debate Keeps Getting Wrong." *IBM Newsroom*, newsroom.ibm.com/blog-lost-in-translation-what-the-ai-code-debate-keeps-getting-wrong. Accessed 11 Mar. 2026.

Sabrina. "Legacy Software Modernization in 2025: Survey of 500+ U.S.. It Pros." *Saritasa*, 25 Aug. 2025, www.saritasa.com/insights/legacy-software-modernization-in-2025-survey-of-500-u-s-it-pros.

"The Dark Side of AI: Assessing Liability When Bots Behave Badly." @Ebglaw, 22 Sept. 2025, www.ebglaw.com/insights/publications/the-dark-side-of-ai-assessing-liability-when-bots-behave-badly.

Appendix User Stories -

LEGACY LIFT

Frontend
React + Vite — 6 DONE

- As a user, I want to **upload programs, copybooks, and data files** into one project, so they convert together.
- As a user, I want my **COBOL beside the generated Python** with review notes, so I can check each line.
- As a user, I want to **reopen a saved project**, so I can continue where I left off.
- As a user, I want my **last translation back after a reload**, so I don't lose my work.
- As a user, I want to **download each Python file**, so I can use it outside LegacyLift.
- As a user, I want **errors in plain words**, so I know what to fix.

Backend
FastAPI — 6 DONE

- As a user, I want to **send COBOL and get draft Python back**, so I can see the modern version.
- As a reviewer, I want **untranslated lines kept as review notes**, so nothing is guessed.
- As a user, I want **every upload checked** for type, size, and encoding, so bad files never reach the converter.
- As a user, I want to **list my projects and delete** the ones I don't need, so my workspace stays tidy.
- As a user, I want to **see which files copy and call each other**, so I know what to convert together.
- As a user, I want to **know if the Python prints the same as the COBOL**, so I can trust the conversion.

Database
PostgreSQL on Neon — 6 DONE

- As a user, I want my **projects saved in a database**, so they survive a redeploy.
- As a user, I want **every uploaded file stored with its project**, so I can read it back.
- As a user, I want **each conversion kept as a run**, so I can reopen the same result later.
- As a user, I want the **generated Python and its review notes saved**, so nothing is converted twice.
- As a user, I want **short project IDs that are never reused**, so links stay correct.
- As a developer, I want the **tables created at startup**, so a new database needs no setup.

Done and merged to **main** — Also done: automated tests, lint, and type checks on every backend change